In [1]:
```
# Pima Indians Diabetes


import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.preprocessing import StandardScaler
```

In [2]:
```
# 1. Load Dataset


df = pd.read_csv("diabetes.csv")   # Update path if needed

print("First 5 rows:")
print(df.head())

print("\nDataset Info:")
print(df.info())

print("\nStatistical Summary:")
print(df.describe())
```

```
First 5 rows:
   Pregnancies  Glucose  BloodPressure  SkinThickness  Insulin   BMI  \
0            6      148             72             35        0  33.6
1            1       85             66             29        0  26.6
2            8      183             64              0        0  23.3
3            1       89             66             23       94  28.1
4            0      137             40             35      168  43.1

   DiabetesPedigreeFunction  Age  Outcome
0                     0.627   50        1
1                     0.351   31        0
2                     0.672   32        1
3                     0.167   21        0
4                     2.288   33        1

Dataset Info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 768 entries, 0 to 767
Data columns (total 9 columns):
 #   Column                    Non-Null Count  Dtype
---  ------                    --------------  -----
 0   Pregnancies               768 non-null    int64
 1   Glucose                   768 non-null    int64
 2   BloodPressure             768 non-null    int64
 3   SkinThickness             768 non-null    int64
 4   Insulin                   768 non-null    int64
 5   BMI                       768 non-null    float64
 6   DiabetesPedigreeFunction  768 non-null    float64
 7   Age                       768 non-null    int64
 8   Outcome                   768 non-null    int64
dtypes: float64(2), int64(7)
memory usage: 54.1 KB
None

Statistical Summary:
       Pregnancies     Glucose  BloodPressure  SkinThickness     Insulin  \
count   768.000000  768.000000     768.000000     768.000000  768.000000
mean      3.845052  120.894531      69.105469      20.536458   79.799479
std       3.369578   31.972618      19.355807      15.952218  115.244002
min       0.000000    0.000000       0.000000       0.000000    0.000000
25%       1.000000   99.000000      62.000000       0.000000    0.000000
50%       3.000000  117.000000      72.000000      23.000000   30.500000
75%       6.000000  140.250000      80.000000      32.000000  127.250000
max      17.000000  199.000000     122.000000      99.000000  846.000000

              BMI  DiabetesPedigreeFunction         Age     Outcome
count  768.000000                768.000000  768.000000  768.000000
mean    31.992578                  0.471876   33.240885    0.348958
std      7.884160                  0.331329   11.760232    0.476951
min      0.000000                  0.078000   21.000000    0.000000
25%     27.300000                  0.243750   24.000000    0.000000
50%     32.000000                  0.372500   29.000000    0.000000
75%     36.600000                  0.626250   41.000000    1.000000
max     67.100000                  2.420000   81.000000    1.000000
```

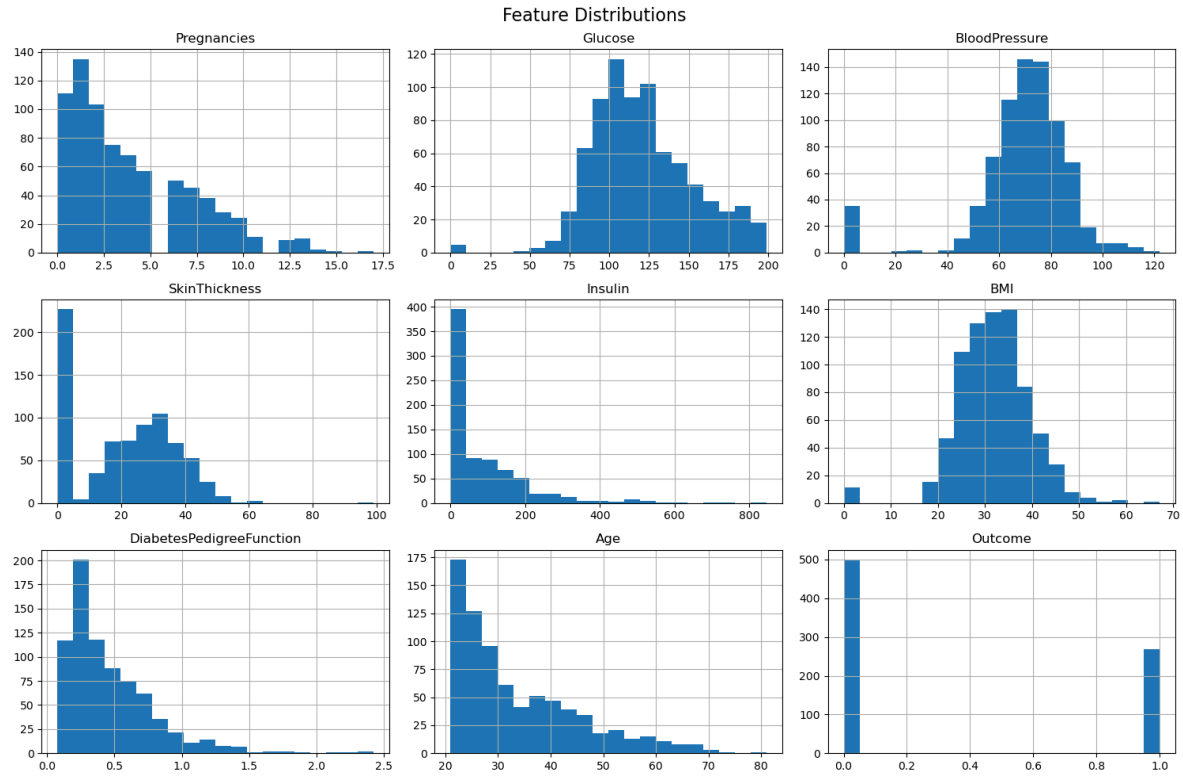
In [3]:
```
# 2. Feature Ranges


print("\nFeature Ranges:")
for col in df.columns:
    print(f"{col}: Min={df[col].min()}, Max={df[col].max()}")
```

```
Feature Ranges:
Pregnancies: Min=0, Max=17
Glucose: Min=0, Max=199
BloodPressure: Min=0, Max=122
SkinThickness: Min=0, Max=99
Insulin: Min=0, Max=846
BMI: Min=0.0, Max=67.1
DiabetesPedigreeFunction: Min=0.078, Max=2.42
Age: Min=21, Max=81
Outcome: Min=0, Max=1
```

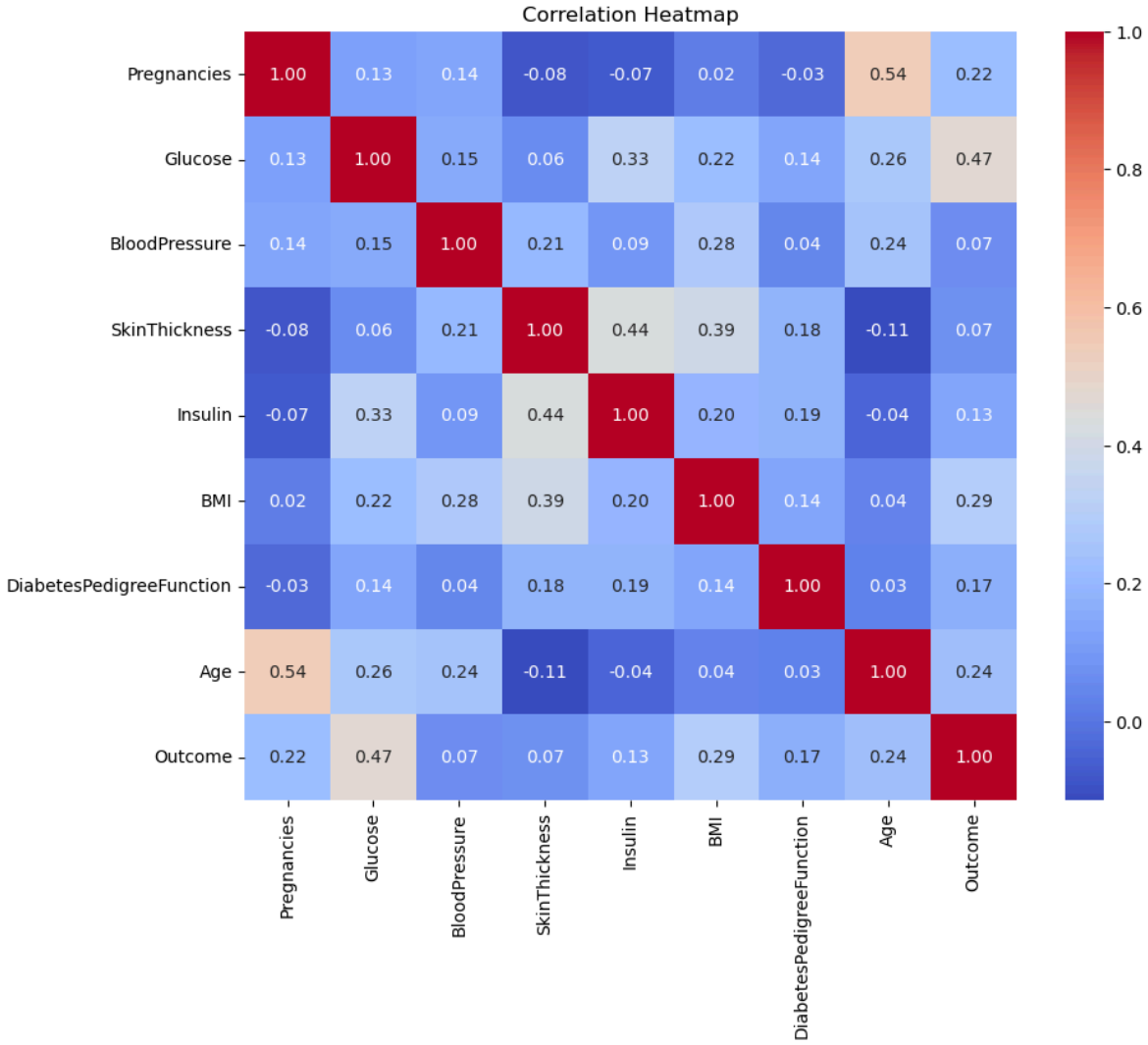
In [4]:
```
# 3. Histograms (All in One Figure)


df.hist(figsize=(15,10), bins=20)
plt.suptitle("Feature Distributions", fontsize=16)
plt.tight_layout()
plt.show()
```



In [5]:
```
# 4. Correlation Analysis


plt.figure(figsize=(10,8))
sns.heatmap(df.corr(), annot=True, cmap="coolwarm", fmt=".2f")
plt.title("Correlation Heatmap")
plt.show()
```



In [6]:
```
# 5. Handling Zero Values (Optional but Recommended)
# Some features should not realistically contain zeros


columns_with_zero_issue = ["Glucose", "BloodPressure",
                           "SkinThickness", "Insulin", "BMI"]

for col in columns_with_zero_issue:
    df[col] = df[col].replace(0, np.nan)
    df[col].fillna(df[col].median(), inplace=True)

print("\nAfter Handling Zero Values:")
print(df.describe())
```

```
After Handling Zero Values:
       Pregnancies     Glucose  BloodPressure  SkinThickness     Insulin  \
count   768.000000  768.000000     768.000000     768.000000  768.000000
mean      3.845052  121.656250      72.386719      29.108073  140.671875
std       3.369578   30.438286      12.096642       8.791221   86.383060
min       0.000000   44.000000      24.000000       7.000000   14.000000
25%       1.000000   99.750000      64.000000      25.000000  121.500000
50%       3.000000  117.000000      72.000000      29.000000  125.000000
75%       6.000000  140.250000      80.000000      32.000000  127.250000
max      17.000000  199.000000     122.000000      99.000000  846.000000

              BMI  DiabetesPedigreeFunction         Age     Outcome
count  768.000000                768.000000  768.000000  768.000000
mean    32.455208                  0.471876   33.240885    0.348958
std      6.875177                  0.331329   11.760232    0.476951
min     18.200000                  0.078000   21.000000    0.000000
25%     27.500000                  0.243750   24.000000    0.000000
50%     32.300000                  0.372500   29.000000    0.000000
75%     36.600000                  0.626250   41.000000    1.000000
max     67.100000                  2.420000   81.000000    1.000000
```

In [7]:
```
# 6. Prepare X and y


X = df.drop("Outcome", axis=1)
y = df["Outcome"]
```

In [8]:
```
# 7. Standardization


scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

print("\nFinal X shape:", X_scaled.shape)
print("Final y shape:", y.shape)

print("\nNo dummy variables required (all features are numeric).")
```

```
Final X shape: (768, 8)
Final y shape: (768,)

No dummy variables required (all features are numeric).
```

In [ ]:

In [ ]:

In [ ]: